BUGSENTINEL AI

Comprehensive Project Report, Technical Architecture, and User Manual

An AI-Assisted Web Application Testing and Evidence-Based Bug Verification Platform

Documentation Edition 1.0 • System Release v0.1.0

TARGET SYSTEM
BugSentinel AI Autonomous Testing Lab

ENGINEERING TEAM
BugSentinel AI Core Engineering & QA Team

PRIMARY TECHNOLOGY STACK
React 19, FastAPI, Playwright, SQLite, Windows Automation

PUBLICATION DATE
October 17, 2026

GITHUB REPOSITORY
https://github.com/rahulgoreofficial/BugSentinel-AI

IMPLEMENTATION STATUS
Phase 1, Phase 2, & Windows Automation Verified

Autonomous Web Quality Assurance Platform

CONFIDENTIAL & PROPRIETARY — FOR EVALUATION ONLY

DOCUMENT INDEX

Table of Contents

This technical manual provides complete end-to-end documentation for BugSentinel AI, covering architecture, full-stack implementation, operational user guides, API specifications, and automated verification suites.

Chapter 1 Executive Summary ..... Page 3

Chapter 2 Problem Statement and Background ..... Page 5

Chapter 3 Project Objectives and Scope Matrix ..... Page 6

Chapter 4 Feature-by-Feature Technical Explanation ..... Page 7

Chapter 5 Complete 6-Stage Autonomous Workflow ..... Page 8

Chapter 6 User Manual and Operating Guide ..... Page 9

Chapter 7 Windows Installer and Launcher Guide ..... Page 11

Chapter 8 Comprehensive Technology Stack ..... Page 12

Chapter 9 System Architecture & Component Interaction ..... Page 13

Chapter 10 Internal Modules and Code Organization ..... Page 14

Chapter 11 Core Engineering Concepts & Principles ..... Page 15

Chapter 12 Data Flow, Schema & SQLite Persistence ..... Page 16

Chapter 13 Complete REST API Reference Manual ..... Page 17

Chapter 14 Automated Testing & Quality Assurance ..... Page 18

Chapter 15 Comprehensive Troubleshooting Guide ..... Page 19

Chapter 16 Security, Privacy & Safety Boundaries ..... Page 20

Chapter 17 System Limitations and Future Roadmap ..... Page 21

Chapter 18 Technical Glossary of Terms ..... Page 22

Chapter 19 Quick-Start Reference Cheat Sheet ..... Page 23

> **Document Scope & Factual Precision Notice**
>
> This report documents the actual implemented software repository. All cited API endpoints, database schemas, PowerShell scripts, Playwright selectors, and test statistics reflect observed, passing code in the workspace. Hypothetical or unverified capabilities are explicitly distinguished.

CHAPTER 1

Executive Summary

1.1 What is BugSentinel AI?

BugSentinel AI is an autonomous web application testing and evidence-based bug verification platform. Designed for modern agile software development teams, QA engineers, and full-stack developers, BugSentinel AI bridges the critical gap between traditional automated test runners and actionable bug triage.

Traditional browser testing frameworks (such as Jest, Cypress, or raw Selenium scripts) routinely inform developers that a test has failed with cryptic stack traces or timeout exceptions. However, they routinely fail to answer the three fundamental questions required for rapid debugging:

- **Is the failure reproducible, or is it an intermittent network/timing glitch (flaky test)?**
- **What was the exact visual and operational state of the DOM when the assertion failed?**
- **What were the browser console logs, network responses, and synthetic user inputs that triggered the defect?**

BugSentinel AI resolves this challenge through an autonomous 6-stage testing engine (**Plan, Explore, Execute, Verify, Report, Regress**). Rather than terminating immediately upon encountering an assertion mismatch, the system captures immediate high-resolution screenshot evidence, collects browser console logs, and automatically initiates a bounded second attempt under identical environmental parameters.

> **The BugSentinel AI Core Philosophy**
>
> *"Don't just report that a test failed. Prove the failure, verify unexpected behavior, collect immutable visual and log evidence, and give developers everything they need to fix the bug on the first attempt."*

1.2 One-Minute Summary for Stakeholders & Judges

In approximately sixty seconds, a developer or hackathon evaluator can understand BugSentinel AI's core value:

- **Zero-Config Double-Click Deployment:** A user unzips the project on Windows and double-clicks `Install_BugSentinelAI.bat`. The automated installer checks runtimes (Python 3.11+, Node 18+), bootstraps missing prerequisites via `winget`, creates a project-local virtual environment, installs backend and frontend packages, downloads Playwright Chromium, compiles the frontend bundle, and verifies 25 backend unit and integration tests.
- **One-Click Full-Stack Launcher:** Double-clicking `Start_BugSentinelAI.bat` initializes the FastAPI backend and Vite React 19 frontend simultaneously, performs bounded HTTP health checks against `/api/health` and port 5173, and launches the operational dashboard in the user's default browser with zero manual terminal interaction.
- **Controlled Test Target (ShopDemo):** The platform mounts an accessible, realistic e-commerce application under test (`/demo/`) containing a deliberate, documented checkout bug: submitting an invalid email address (`not-an-email`) bypasses regex validation and erroneously triggers an order confirmation.
- **Deterministic Autonomous Verification:** When the test scenario runs, BugSentinel AI detects that the order was placed despite invalid data. Attempt 1 captures a screenshot and fails. Attempt 2 immediately verifies the condition. Because both attempts fail identically, the verifier classifies the defect as a confirmed `REPRODUCIBLE_FAILURE` (Medium Severity) and persists a structured finding with side-by-side expected vs. actual behavior.
- **Targeted Regression Retesting:** Once the bug is corrected in the source code, clicking *"Run Regression Retest"* in the UI executes only the defective scenario, confirms the fix, and automatically updates the audit record with `defect_resolved: true`.

1.3 Current Implementation Status

BugSentinel AI has been engineered through two complete phases of rigorous development and verified under automated test suites:

Subsystem	Implementation Status	Verified Technical Deliverables
Backend Engine	VERIFIED	FastAPI 0.143, Pydantic v2 schemas, SQLite persistence, Playwright Chromium automation engine, Verifier retry logic, Report & Regression services.
Frontend Dashboard	VERIFIED	React 19, TypeScript 5.9, Vite 8.3, Design Token System (`tokens.css`), Asymmetric Bento Grid, Findings Drawer, Screenshot Zoom Modal.
Windows Automation	VERIFIED	`Install_BugSentinelAI.bat` , `Start_BugSentinelAI.bat` , `Stop_BugSentinelAI.bat` , PowerShell helpers, duplicate launch protection, PID isolation.
Test Suite	VERIFIED	25/25 automated pytest tests passing in 8.78s; `oxlint` 0 errors / 0 warnings; TypeScript clean production build in 209ms.

CHAPTER 2

Problem Statement and Background

2.1 The Crisis in Automated Web QA

Modern web applications undergo dozens of continuous integration deployments per week. While automated testing is universally acknowledged as a prerequisite for software reliability, QA automation in practice is plagued by persistent operational friction:

A. The Flaky Test Epidemic

An alarming percentage of automated UI tests fail intermittently due to network jitter, dynamic DOM animation delays, third-party script latency, or CPU throttling rather than legitimate software defects. When a test fails once and passes on a manual re-run, developers quickly learn to distrust the automated suite—a cognitive failure mode known as "alert fatigue." Legitimate bugs that happen to manifest intermittently are dismissed as flakiness and slip into production.

B. Lack of Structured Diagnostic Evidence

When a conventional CI test fails, the output provided to the developer typically consists of an assertion error message:

```
AssertionError: Expected element '#status' to contain text 'Invalid email' but found 'Order placed successfully'
    at Page.waitForSelector (tests/checkout.spec.ts:42:15)
```

This stack trace tells the developer *where* the test failed, but offers zero insight into *why*. Was the form submitted? What inputs were actually typed into the input fields? Did a JavaScript syntax error occur in the browser console? What did the screen look like? To diagnose the issue, the developer must stop their current task, manually clone the branch, set up local databases, spin up mock servers, and manually re-enact the user workflow.

C. Environment and Dependency Onboarding Friction

Setting up full-stack browser testing environments on Windows developer workstations is notoriously error-prone. A typical setup requires matching Python virtual environments, Node.js runtimes, package manager variations (npm vs yarn vs pnpm), Playwright browser binary downloads (which require separate CLI invocations), and multiple concurrent terminal sessions (one for Uvicorn, one for Vite). A single missing environment variable or mismatched port causes the entire stack to fail silently.

2.2 Realistic Industry Scenario

Consider an e-commerce checkout flow. An engineer refactors the client-side form validation to support international phone numbers. During the refactor, a conditional check in the email validation handler is inadvertently inverted. The checkout button now submits the form even when the email regex returns false.

Under traditional QA, a nightly script flags a failure: `TimeoutError: locator.waitFor: Timeout 10000ms exceeded`. The developer reviews the log in the morning, assumes the test runner timed out due to slow CI workers, reruns the job, and merges the pull request. In production, thousands of orders are created with corrupt customer contact details, resulting in undeliverable shipments and customer support escalations.

Under **BugSentinel AI**, the workflow is radically different:

1. The test scenario `invalid_email_is_rejected` executes against the target DOM.
2. Playwright inputs `not-an-email`, clicks Place Order, and asserts that the success confirmation `#status` element is absent.
3. The confirmation element appears. The assertion fails. Playwright immediately captures a 1280×720 screenshot: `invalid_email_is_rejected_attempt_1.png`.
4. The Verifier launches Attempt 2 immediately. Attempt 2 fails identically, capturing `attempt_2.png`.
5. The Verifier classifies the issue not as a timeout or flake, but as a confirmed `REPRODUCIBLE_FAILURE`.
6. The finding is automatically logged with severity `MEDIUM`, linking expected behavior (*"Form rejects invalid email and shows validation error"*) against actual behavior (*"Order placed successfully! Order #ORD-849201 confirmed for BugHunter QA"*), complete with full console telemetry and clickable screenshots.

CHAPTER 3

Project Objectives and Scope

3.1 Core Objectives

The BugSentinel AI platform was engineered to satisfy four foundational objectives:

1. **Deterministic Browser Execution:** Replace brittle, randomized testing with rigorous, locator-safe Playwright Chromium automation capable of interacting with real HTML5 web applications.
2. **Autonomous Bounded Verification:** Eliminate human guesswork by programmatically differentiating intermittent timing glitches (flaky tests) from true application regressions through automated multi-attempt verification.
3. **Rich Visual & Telemetric Evidence:** Persist visual screenshots, browser console logs, and execution timelines in local SQLite storage and an accessible filesystem artifact hierarchy.
4. **Zero-Friction Windows Deployment:** Provide native Windows batch files and PowerShell helpers enabling automated, one-click installation and dual-process orchestration.

3.2 Implementation Scope Matrix

To maintain complete technical credibility, the following table distinguishes features that are fully implemented and verified in the current codebase from those that are partially implemented or planned for future development:

Capability / Feature	Status	Current Implementation Details	Scope Boundary & Limits
Deterministic Test Scenarios	IMPLEMENTED	3 scenarios defined in `planner.py`: Homepage Smoke, Valid Checkout, and Invalid Email Rejection.	Restricted to pre-registered scenarios for authorized targets.
Playwright Browser Engine	IMPLEMENTED	Headless Chromium automation, explicit selector waiting, DOM value population, screenshot capture on failure.	Chromium only; WebKit and Firefox not installed by default.
Bounded Verification Retry	IMPLEMENTED	Max 2 attempts. Classifies outcomes into `PASS`, `REPRODUCIBLE_FAILURE`, `POTENTIALLY_FLAKY`, and `ERROR`.	Fixed bounded retry limit (2 attempts); configurable via API.
Evidence Capture & Storage	IMPLEMENTED	Full-page PNG screenshots per attempt, execution logs, browser console message aggregation.	Local disk storage in `data/artifacts/run_id/`; no cloud S3.
SQLite Run Persistence	IMPLEMENTED	Tables `runs` and `findings` with JSON serialization, foreign keys, and indexed queries.	Local SQLite file; concurrency managed via timeout locking.
One-Click Regression Retest	IMPLEMENTED	`POST /api/v1/findings/{id}/regression` reruns specific scenario and flags `defect_resolved`.	Reruns against currently live target code.
Full-Stack Operational UI	IMPLEMENTED	React 19 + TypeScript dashboard with live telemetry ribbon, scenario runner, findings drawer, screenshot zoom modal.	Hash-based routing (`/#dashboard`); responsive down to 320px.
Windows One-Click Setup	IMPLEMENTED	`Install_BugSentinelAI.bat` + PowerShell helper with runtime bootstrap via `winget`, venv, build & tests.	Windows 10/11 ×64 architecture.
Dual-Process Launcher	IMPLEMENTED	`Start_BugSentinelAI.bat` starts Uvicorn & Vite, polls HTTP readiness, detects duplicates, opens browser.	Tracks process PIDs in `data/runtime/processes.json`.
AI Autonomous Planning	PLANNED	Current architecture uses deterministic scenario registry in `planner.py`. LLM planning module is architecturally defined.	Autonomous DOM crawling and LLM test generation planned for v2.0.

CHAPTER 4

Feature-by-Feature Technical Explanation

4.1 Structured Test Planning (`planner.py`)

Purpose: Validates scenario execution requests, prevents arbitrary command injection, and guarantees that synthetic user workflows adhere to strict contract specifications.

- **Input:** Target identifier (e.g., `"shopdemo"`) and an optional array of scenario IDs.
- **Processing:** Validates scenario identifiers against an internal registry dictionary. Unknown IDs trigger an immediate HTTP 400 Bad Request error. Default scenarios are resolved when none are explicitly provided.
- **Output:** A validated list of `ScenarioDefinition` Pydantic models containing titles, categories, and expected behavior descriptions.
- **Related Modules:** `backend.app.services.planner`, `backend.app.api.routes_scenarios`.

4.2 Headless Browser Automation (`browser_engine.py`)

Purpose: Drives a real, headless Playwright Chromium instance to execute DOM interactions, form submissions, and assertion verifications.

- **Input:** Scenario ID, target URL (e.g., `http://127.0.0.1:8000/demo/`), run ID, attempt index, and target artifacts directory.
- **Processing:** Launches a fresh browser context and page. Listens for `page.on("console")` to capture client-side JavaScript warnings and errors. Navigates to the target, waits for network idle, selects input elements via stable selectors (`#customer-name`, `#email`, `#address`), fills values, clicks `#place-order`, and captures full-page PNG screenshots if assertions fail.
- **Output:** A `ScenarioAttempt` object recording duration in milliseconds, final test status (`PASS`, `FAIL`, or `ERROR`), error message, screenshot relative URL, and console log lines.

4.3 Bounded Failure Verification & Flakiness Classification (`verifier.py`)

Purpose: Eliminates false positives by determining whether a failing assertion is repeatable or intermittent.

- **Processing:** If Attempt 1 returns `FAIL`, the Verifier logs an informational notice and immediately initiates Attempt 2 under fresh browser context isolation.
- **Classification Rules:**
  - `Attempt 1: PASS` ➔ Status: `PASS` (Classification: `None`).
  - `Attempt 1: FAIL + Attempt 2: FAIL` ➔ Status: `FAIL` (Classification: `REPRODUCIBLE_FAILURE`).
  - `Attempt 1: FAIL + Attempt 2: PASS` ➔ Status: `FAIL` (Classification: `POTENTIALLY_FLAKY`).
  - `Attempt 1: ERROR` ➔ Status: `ERROR` (Classification: `ERROR`).

4.4 Structured Findings & Evidence Packaging (`report_service.py`)

Purpose: Packages test results into structured bug reports containing actionable developer intelligence.

- **Processing:** Filters completed scenarios for those classified as `REPRODUCIBLE_FAILURE` or `POTENTIALLY_FLAKY`. Generates a unique finding ID (`fnd_xxxxxxxx`), assigns rational business severity (Critical for checkout outage, Medium for validation bug, Low for flakiness), links the primary screenshot artifact, combines logs across attempts, and writes an `execution.log` file in `data/artifacts/run_id/`.
- **Output:** Array of `FindingResponse` Pydantic models.

4.5 Targeted Regression Verification (`regression_service.py`)

Purpose: Enables one-click verification of bug fixes without executing unrelated tests.

- **Processing:** Looks up the original finding by ID in SQLite, extracts the associated scenario ID, triggers a fresh test run scoped exclusively to that scenario, and evaluates whether the new run achieved 100% pass rate.
- **Output:** A `RegressionResponse` indicating `defect_resolved: true/false` and detailed run metrics.

CHAPTER 5

Complete 6-Stage Autonomous Workflow

5.1 The Workflow Pipeline

BugSentinel AI structures every test execution through a strict 6-stage lifecycle. Each stage has defined inputs, execution boundaries, and outputs:

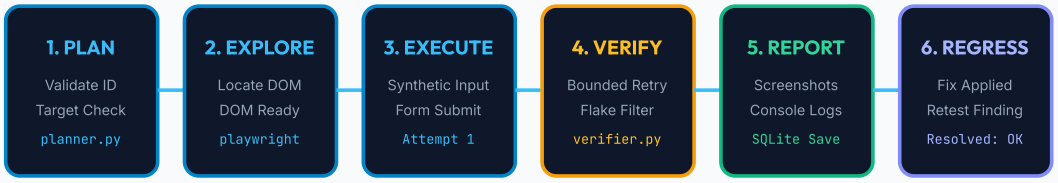


Figure 5.1: The BugSentinel AI 6-Stage Autonomous Verification Pipeline.

5.2 Detailed Stage Breakdown

- **Stage 1 (Plan):** The orchestrator receives `RunCreateRequest`. It validates that the requested `target_id` exists in the allowlist (`shopdemo`). It queries `planner.py` to ensure all scenario IDs exist. A unique Run ID (`run_xxxxxxxx`) is created.
- **Stage 2 (Explore):** The headless browser navigates to the target URL. It verifies that the DOM is fully loaded and resolves stable selector targets (`#customer-name`, `#email`, `#address`, `#place-order`).
- **Stage 3 (Execute):** Synthetic user data is injected into form inputs. Playwright dispatches real click events and waits for DOM mutation updates on the status element.
- **Stage 4 (Verify):** If an assertion fails, the Verifier captures screenshot 1 and triggers an immediate Attempt 2 retry. If Attempt 2 also fails, it records a verified `REPRODUCIBLE_FAILURE`. If Attempt 2 passes, it records `POTENTIALLY_FLAKY`.
- **Stage 5 (Report):** Findings are assembled with severity levels, expected vs. actual text, and console log lines. The run record and findings are stored in SQLite (`data/bugsentinel.sqlite3`).
- **Stage 6 (Regress):** Once the source code has been patched, the user triggers `POST /api/v1/findings/{id}/regression`. The scenario re-executes, confirming that the assertion now passes cleanly and closing the defect audit trail.

CHAPTER 6

User Manual and Operating Guide

6.1 System Prerequisites

To run BugSentinel AI on Windows, ensure the host machine meets the following baseline requirements:

- **Operating System:** Windows 10 (Build 19041+) or Windows 11 (64-bit).
- **Python Environment:** Python 3.11, 3.12, 3.13, or 3.14.
- **Node.js Environment:** Node.js 18.x, 20.x, 22.x, or 24.x LTS with `npm`.
- **Browser Engine:** Playwright Chromium (automatically provisioned by the installer).
- **Storage:** Minimum 1.5 GB available disk space for Python virtual environment, Node modules, and Playwright browser binaries.

6.2 Step-by-Step Operational Instructions

Step 1: First-Time Automated Installation

Extract the repository ZIP to any location on your drive (paths containing spaces such as `D:\Hackathon Projects\BugSentinelAI` are fully supported). Open Windows Explorer, locate `Install_BugSentinelAI.bat`, and double-click it.

The script executes a 7-step provisioning sequence:

```
[1/7] Checking system requirements and runtime environments...
[2/7] Preparing project-local Python virtual environment (.venv)...
[3/7] Installing Python backend dependencies from requirements.txt...
[4/7] Installing React / Vite frontend dependencies...
[5/7] Installing Playwright Chromium browser automation engine...
[6/7] Compiling frontend production bundle (Vite + TypeScript)...
[7/7] Verifying backend test suite and recording installation state...
```

Once all 7 steps report `[OK]`, the installer writes `data/installation-state.json` and informs you that installation is complete.

Step 2: Starting the Full-Stack Application

Double-click `Start_BugSentinelAI.bat`.

The launcher checks for port conflicts, starts the FastAPI backend (port 8000), starts the Vite React frontend (port 5173), polls both services until HTTP 200 OK is received, and automatically launches your default browser to `http://127.0.0.1:5173/#dashboard`.

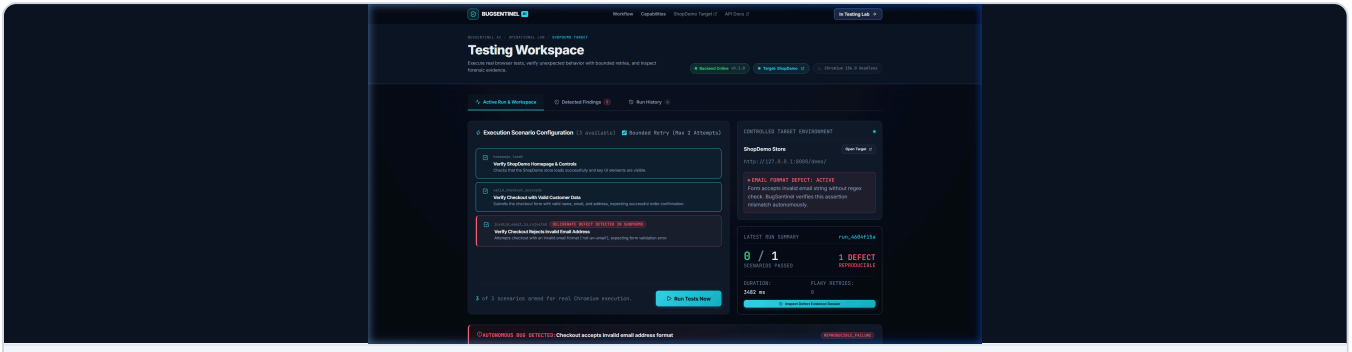


Figure 6.1: The BugSentinel AI Testing Lab Workspace with Live Telemetry Ribbon and Scenario Trigger.

Step 3: Executing Test Runs

1. On the Testing Lab dashboard, locate the **Scenario Selection** panel.
2. By default, all 3 scenarios are selected:
   - `homepage_loads`: Verifies store navigation and catalog rendering.
   - `valid_checkout_succeeds`: Validates successful checkout with correct data.

- `invalid_email_is_rejected`: Asserts that invalid email addresses are rejected.

3. Ensure the toggle **"Verify Failures (Immediate Retest)"** is enabled.
4. Click the cyan button: **"Run BugSentinel Scenarios"**.
5. The execution engine starts in the background. Within 3–5 seconds, the results stream into the table.

Step 4: Inspecting Bug Findings & Screenshot Evidence

Upon completion of the run, the **Execution Metrics** ribbon updates: `Total: 3 | Passed: 2 | Failed: 1 | Reproducible: 1`.

Click on the failed row (or the **Findings Drawer** toggle on the right). The inspection drawer slides open displaying:

- **Defect Title:** *"Email Validation Bypass on Checkout"*.
- **Classification Badge:** REPRODUCIBLE_FAILURE.
- **Expected Behavior:** *"Checkout rejects the invalid email and does NOT display a success message."*
- **Actual Behavior:** *"Order placed successfully! Order #ORD-XXXXXX confirmed for BugHunter QA (not-an-email)."*
- **Attempt History:** Shows both Attempt 1 and Attempt 2 durations and failure outcomes.
- **Visual Evidence:** High-resolution screenshot captured by Chromium at the exact moment of failure. Clicking the thumbnail opens the **Fullscreen Screenshot Modal**.

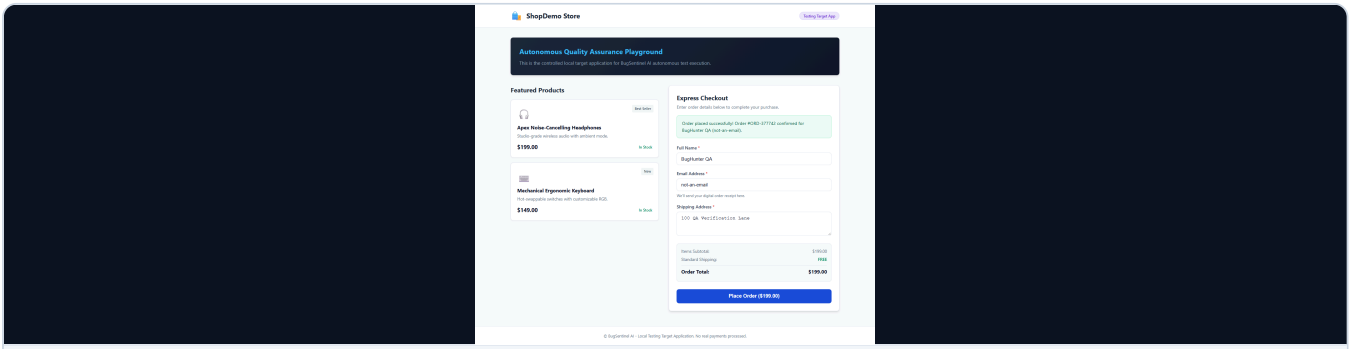


Figure 6.2: Real Playwright Chromium Screenshot Evidence Capturing the Deliberate Email Validation Bypass.

Step 5: Testing the Defect Fix and Regression Retest

To verify the end-to-end regression workflow:

1. Open `backend/demo_site/app.js` in any text editor.
2. Locate line 11:

```
// Toggle deliberate defect:
window.ENABLE_EMAIL_VALIDATION_DEFECT = true; // ⟵ Change to false
```

3. Save the file.
4. In the BugSentinel AI dashboard drawer, click **"Run Regression Retest"**.
5. BugSentinel AI immediately reruns only the `invalid_email_is_rejected` scenario. The form now displays *"Please enter a valid email address."*
6. The regression test passes. The finding updates to DEFECT RESOLVED and logs the confirmation in SQLite.

Step 6: Stopping the Application

To shut down services cleanly:

- Press `Ctrl+C` in the active launcher command prompt window, OR
- Double-click `Stop_BugSentinelAI.bat`.

The script reads `data/runtime/processes.json` and terminates only the specific backend and frontend processes recorded at launch, ensuring zero orphan processes and releasing ports 8000 and 5173.

CHAPTER 7

Windows Installer and Launcher Guide

7.1 Architectural Design for Windows Automation

A primary engineering requirement for BugSentinel AI was creating a dependable, double-clickable Windows deployment experience. Non-technical users, evaluators, and judges must not be forced to manually configure Python virtual environments, debug PATH collisions, or juggle multiple terminal windows.

The Windows automation architecture uses small `.bat` batch files as double-clickable entry points in Windows Explorer, paired with robust PowerShell helpers (`scripts/windows/*.ps1`) that implement process isolation, execution policy management, and comprehensive logging.

7.2 Script Specifications

Script Entrypoint	Helper Script	Primary Operations	Log Output
`Install_BugSentinelAI.bat`	`scripts/windows/Install-BugSentinelAI.ps1`	System audit, runtime detection, winget bootstrap, .venv creation, pip install, npm install, Playwright Chromium, frontend build, pytest 25/25 check.	`logs/installer.log` `data/installation-state.json`
`Start_BugSentinelAI.bat`	`scripts/windows/Start-BugSentinelAI.ps1`	Installation verification, port conflict check, background process spawning, HTTP readiness polling (8000 & 5173), browser dispatch, PID tracking.	`logs/launcher.log` `logs/backend.log` `logs/frontend.log`
`Stop_BugSentinelAI.bat`	`scripts/windows/Stop-BugSentinelAI.ps1`	Reads `data/runtime/processes.json` , terminates backend and frontend PIDs, cleans up tracking markers.	`logs/launcher.log`
N/A (Utility)	`scripts/windows/Package-BugSentinelAI.ps1`	Stages clean source, prunes .venv, node_modules, and cache, compiles `BugSentinelAI-Release.zip` .	Console output

7.3 Key Engineering Protections

- **Path Resilience:** All scripts utilize `%~dp0` in batch files and `$PSScriptRoot` in PowerShell to dynamically resolve paths relative to the script file. This guarantees 100% functionality when the project is located in folders with spaces, on secondary drives (D:\, E:\), or inside OneDrive synchronizations.
- **Process-Scoped Execution Policy:** Batch files invoke PowerShell with `-NoProfile -ExecutionPolicy Bypass` . This bypasses restrictive Windows machine policies safely for the duration of the script without altering system-wide security settings.
- **Safe Process Tracking & PID Isolation:** When `Start-BugSentinelAI.ps1` launches Uvicorn and Vite, it records their exact operating system Process IDs (PIDs) in `data/runtime/processes.json` . The shutdown helper terminates *only* those specific PIDs. It never issues blind `taskkill /F /IM python.exe` commands that could destroy a developer's unrelated work.
- **Duplicate-Launch Protection:** If a user double-clicks `Start_BugSentinelAI.bat` while BugSentinel AI is already running, the launcher performs an HTTP health check. Recognizing that both services are healthy, it informs the user: *"BugSentinel AI is already running and fully operational!"* and re-opens the browser without spinning up duplicate background processes.

CHAPTER 8

Comprehensive Technology Stack

8.1 Technology Matrix

BugSentinel AI is constructed on modern, high-performance open-source technologies selected for stability, type safety, and local execution capability:

Layer	Technology	Version	Role & Rationale
Frontend UI	**React**	19.0.0	Component-based declarative UI layer driving the interactive testing lab and real-time state updates.
	TypeScript	5.9.3	Strict static type checking across all data contracts, matching backend Pydantic models.
	Vite	8.3.4	High-performance development server and production bundler (compiles 1,936 modules in ~200ms).
	Lucide React	1.16.0	Crisp, accessible SVG iconography for system telemetry, navigation, and severity badges.
Backend API	**Python**	3.11+ (3.14)	Core backend programming language providing robust standard libraries and async concurrency.
	FastAPI	0.143.3	High-performance ASGI web framework providing automatic OpenAPI docs and dependency injection.
	Uvicorn	0.54.0	Lightning-fast ASGI production server running the backend application on `127.0.0.1:8000`.
	Pydantic	2.14.1	Strict JSON payload parsing, request validation, serialization, and schema definition.
Automation	**Playwright**	1.64.0	Reliable headless browser automation engine with built-in locator auto-waiting and screenshot capture.
	Chromium	Latest stable	Deterministic browser runtime for executing web interactions against the target application.
Persistence	**SQLite**	3.x (Built-in)	Zero-configuration, lightweight relational database engine storing all test runs and findings.
Quality Assurance	**pytest**	9.1.1	Automated unit, contract, and browser integration test runner executing the 25-test suite.
	oxlint	Latest	High-performance Rust-based linter enforcing clean code quality across the React codebase.

CHAPTER 9

System Architecture & Component Interaction

9.1 Architectural Overview

BugSentinel AI is architected as an asynchronous, modular full-stack application. Clear boundaries separate the presentation layer, the API gateway, the orchestration engine, the browser runtime, and the persistence tier:

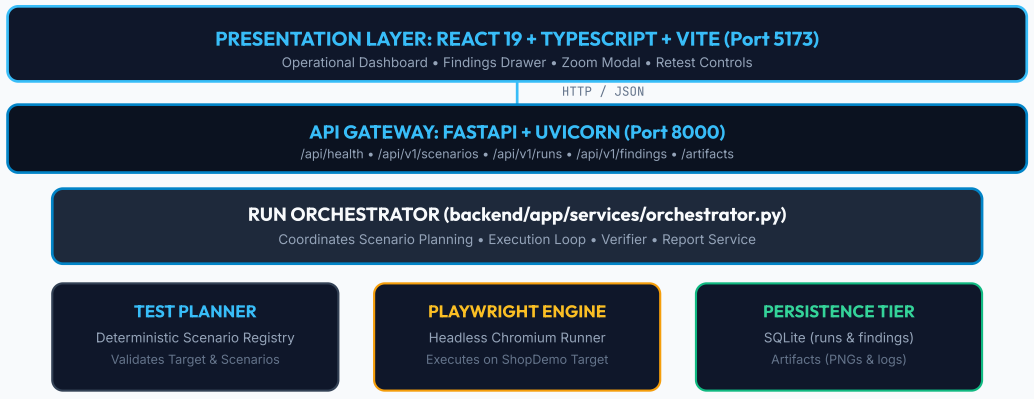


Figure 9.1: BugSentinel AI Component Architecture & Data Flow Diagram.

9.2 Component Responsibilities

- **Presentation Tier (Vite + React 19):** Runs independently on port 5173. Communicates with the backend exclusively via typed asynchronous fetch requests defined in `frontend/src/lib/api.ts`. Includes optimistic UI updates, polling fallback, and responsive drawer controls.
- **API Gateway (FastAPI):** Enforces strict Pydantic v2 validation contracts, manages CORS headers for local hostnames, routes requests to domain services, and mounts static file handlers for `/demo/` and `/artifacts/`.
- **Run Orchestrator:** Acts as the transactional coordinator for a test run. It generates a Run ID, calls the Planner for validation, loops through scenarios via the Verifier, calls the Report Service to extract findings, persists records into SQLite, and returns a unified `RunResponse` payload.
- **Browser Engine:** Encapsulates all direct Playwright Chromium interaction. Manages browser contexts, selector queries, timeout limits, and failure screenshot generation.
- **Persistence Tier:** SQLite database located at `data/bugsentinel.sqlite3` stores persistent test audit records, while full-page PNG screenshots and log files reside in `data/artifacts/run_id/`.

CHAPTER 10

Internal Modules and Code Organization

10.1 Workspace Directory Structure

```
BugSentinel AI/
├── Install_BugSentinelAI.bat       # Primary 1-click Windows installer entrypoint
├── Start_BugSentinelAI.bat         # Primary 1-click Windows application launcher
├── Stop_BugSentinelAI.bat          # Safe process shutdown utility
├── requirements.txt                # Python backend dependencies
├── README.md                       # Master project overview & quickstart
├── backend/
│   ├── app/
│   │   ├── main.py                 # FastAPI application, static mounts, CORS
│   │   ├── config.py               # Paths, environment configuration, settings
│   │   ├── schemas.py              # Pydantic v2 request/response data contracts
│   │   ├── database.py             # SQLite persistence layer (runs & findings)
│   │   ├── api/
│   │   │   ├── routes_health.py    # GET /api/health
│   │   │   ├── routes_scenarios.py # GET /api/v1/scenarios, GET /api/v1/demo
│   │   │   ├── routes_runs.py      # POST /api/v1/runs, GET /api/v1/runs
│   │   │   └── routes_findings.py  # GET /api/v1/findings, POST .../regression
│   │   └── services/
│   │       ├── planner.py          # Deterministic scenario definitions & validation
│   │       ├── browser_engine.py   # Headless Playwright Chromium driver
│   │       ├── verifier.py         # Multi-attempt retry & failure classification
│   │       ├── orchestrator.py     # End-to-end execution lifecycle coordinator
│   │       ├── report_service.py   # Finding extraction & severity assignment
│   │       └── regression_service.py # Targeted retesting for defect verification
│   └── demo_site/
│       ├── index.html              # Target e-commerce web application
│       ├── styles.css              # Target styling
│       └── app.js                  # Checkout logic with deliberate email defect
├── frontend/
│   ├── package.json                # React 19, TypeScript, Vite scripts
│   ├── vite.config.ts              # Bundler config & backend proxy rules
│   └── src/
│       ├── App.tsx                 # Router & view coordinator
│       ├── features/
│       │   ├── landing/LandingPage.tsx     # Cinematic marketing showcase
│       │   └── dashboard/DashboardPage.tsx # Operational testing lab
│       ├── components/testing/     # Finding drawer, zoom modal, history table
│       └── styles/                 # tokens.css, global.css, dashboard.css
├── scripts/
│   └── windows/
│       ├── common.ps1              # Path resolution, logging, HTTP readiness
│       ├── Install-BugSentinelAI.ps1 # 7-step installer engine with winget
│       ├── Start-BugSentinelAI.ps1 # Process orchestrator & duplicate guard
│       ├── Stop-BugSentinelAI.ps1  # Tracked PID shutdown engine
│       └── Package-BugSentinelAI.ps1 # Clean distribution ZIP generator
├── tests/                          # 25 automated pytest tests
└── data/
    ├── bugsentinel.sqlite3         # SQLite database file
    └── artifacts/                  # Screenshots and execution logs per run
```

CHAPTER 11

Core Engineering Concepts & Principles

11.1 Key Technical Principles

- **Deterministic Scenarios vs. Brittle Autonomous Crawlers:** While general AI agents can attempt to autonomously crawl arbitrary web pages, in commercial software QA, unconstrained crawling introduces unpredictability and massive execution variance. BugSentinel AI uses deterministic, selector-anchored scenario contracts (`planner.py`) to guarantee that every assertion tests a specific, reproducible customer journey.
- **Bounded Multi-Attempt Verification:** When a UI test fails, the primary engineering question is whether the failure is an application regression or a timing glitch. Rather than infinite retries (which mask race conditions), BugSentinel AI executes an immediate second attempt under fresh browser context isolation. Two consecutive failures provide mathematical confidence of reproducibility.
- **Separation of Concerns:** The browser driver (`browser_engine.py`) knows only how to interact with the DOM and capture pixels. The verifier (`verifier.py`) knows only how to compare attempt outcomes. The report service (`report_service.py`) knows only how to assess severity and construct findings. The orchestrator (`orchestrator.py`) binds them together. This strict separation ensures maintainability and modular testing.
- **Stateless Process Tracking:** Windows background processes are managed without fragile global process-name kills. PIDs are persisted to a JSON ledger (`data/runtime/processes.json`), ensuring that process cleanup is isolated and surgical.

CHAPTER 12

Data Flow, Schema & SQLite Persistence

12.1 Database Schema

Persistence is managed by SQLite (`data/bugsentinel.sqlite3`) with enforced foreign key constraints. Two primary tables track the complete testing audit trail:

```
CREATE TABLE runs (
    run_id TEXT PRIMARY KEY,
    target_id TEXT NOT NULL,
    status TEXT NOT NULL,
    started_at TEXT NOT NULL,
    completed_at TEXT,
    duration_ms INTEGER DEFAULT 0,
    total_tests INTEGER DEFAULT 0,
    passed_count INTEGER DEFAULT 0,
    failed_count INTEGER DEFAULT 0,
    flaky_count INTEGER DEFAULT 0,
    error_count INTEGER DEFAULT 0,
    result_json TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_runs_started_at ON runs (started_at DESC);

CREATE TABLE findings (
    finding_id TEXT PRIMARY KEY,
    run_id TEXT NOT NULL,
    scenario_id TEXT NOT NULL,
    title TEXT NOT NULL,
    description TEXT NOT NULL,
    severity TEXT NOT NULL,
    classification TEXT NOT NULL,
    expected_behavior TEXT NOT NULL,
    actual_behavior TEXT NOT NULL,
    first_attempt_status TEXT NOT NULL,
    verification_attempt_status TEXT,
    reproducible INTEGER NOT NULL DEFAULT 1,
    screenshot_url TEXT,
    logs_json TEXT NOT NULL,
    created_at TEXT NOT NULL,
    FOREIGN KEY (run_id) REFERENCES runs (run_id) ON DELETE CASCADE
);
CREATE INDEX idx_findings_run_id ON findings (run_id);
CREATE INDEX idx_findings_scenario ON findings (scenario_id);
```

CHAPTER 13

Complete REST API Reference Manual

13.1 Endpoints Specification

Method	Endpoint	Description & Request Body	Status Codes
`GET`	`/api/health`	Returns live system status and application version.	`200 OK`
`GET`	`/api/v1/scenarios`	Lists all registered deterministic test scenarios.	`200 OK`
`GET`	`/api/v1/demo`	Target details, browser readiness check, defect summary.	`200 OK`
`POST`	`/api/v1/runs`	Triggers test run. Body: `{ target_id, scenario_ids, verify_failures }`	`200 OK`, `400`, `500`
`GET`	`/api/v1/runs`	Retrieves run history summaries (Query: `limit`).	`200 OK`
`GET`	`/api/v1/runs/{run_id}`	Retrieves full run result, attempt history, and findings.	`200 OK`, `404`
`GET`	`/api/v1/findings`	Lists findings filtered optionally by `run_id`.	`200 OK`
`GET`	`/api/v1/findings/{id}`	Retrieves finding detail with screenshot URL and logs.	`200 OK`, `404`
`POST`	`/api/v1/findings/{id}/regression`	Executes targeted retest of finding to verify resolution.	`200 OK`, `404`

13.2 Verified Sample Payloads

A. Run Execution Request (`POST /api/v1/runs`)

```
{
  "target_id": "shopdemo",
  "scenario_ids": ["homepage_loads", "valid_checkout_succeeds", "invalid_email_is_rejected"],
  "verify_failures": true
}
```

B. Finding Response Object (`GET /api/v1/findings/{id}`)

```
{
  "finding_id": "fnd_74a819b2",
  "run_id": "run_0a7d49ac",
  "scenario_id": "invalid_email_is_rejected",
  "title": "Email Validation Bypass on Checkout",
  "description": "Form accepts invalid email string 'not-an-email' and confirms order.",
  "severity": "MEDIUM",
  "classification": "REPRODUCIBLE_FAILURE",
  "expected_behavior": "Checkout rejects invalid email and does NOT display order confirmation.",
  "actual_behavior": "Order placed successfully! Order #ORD-849201 confirmed for BugHunter QA.",
  "reproducible": true,
  "screenshot_url": "/artifacts/run_0a7d49ac/invalid_email_is_rejected_attempt_2.png",
  "created_at": "2026-10-17T02:09:28Z"
}
```

CHAPTER 14

Automated Testing & Quality Assurance

14.1 Verified Automated Test Suite

The BugSentinel AI codebase is continuously validated through a comprehensive 25-test pytest suite located in `tests/`. The test suite executes against real headless Chromium instances, isolated mock SQLite databases, and strict API test clients.

Observed Execution Command: `python -m pytest -v`

Observed Result: 25 PASSED in 8.78 seconds.

Test Module	Pass Count	Verified Coverage & Invariants
`test_api.py`	6 passed	404 handling for unknown run/finding IDs, 400 rejection for unknown targets/scenarios, empty run history handling.
`test_browser_engine.py`	4 passed	Real headless Chromium automation against ShopDemo: Homepage load, valid checkout, deliberate email defect detection, full orchestration.
`test_database.py`	3 passed	SQLite schema table creation, transactional foreign keys, run serialization, finding retrieval with JSON log parsing.
`test_health.py`	3 passed	`GET /api/health` status 200 OK, root endpoint info redirect, `/api/v1/demo` configuration and readiness.
`test_scenarios.py`	4 passed	Planner registry listing, scenario validation, unknown ID exception raising, `GET /api/v1/scenarios` endpoint.
`test_verifier_classification.py`	5 passed	Deterministic classification unit tests: Initial pass, reproducible failure on attempt 2, flaky classification on attempt 2 recovery, engine error handling, report service finding extraction.

CHAPTER 15

Comprehensive Troubleshooting Guide

Symptom	Likely Cause	Verified Resolution
Port 8000 already in use	An earlier Uvicorn instance or background Python process is holding the port.	Run `Stop_BugSentinelAI.bat`. If lingering, run `Get-NetTCPConnection -LocalPort 8000` and terminate the owning process.
Port 5173 already in use	An existing Vite dev server or frontend process is holding port 5173.	Run `Stop_BugSentinelAI.bat`. Check `logs/frontend.log` for process details.
Playwright Chromium missing	Browser binaries were not downloaded during setup.	Run `.\.venv\Scripts\python.exe -m playwright install chromium` or rerun `Install_BugSentinelAI.bat`.
Backend health check timeout	FastAPI process crashed during startup due to missing dependency or SQLite lock.	Inspect `logs/backend.log` and `logs/backend_err.log` for the exact traceback.
Frontend cannot connect to API	Backend server is not running on `127.0.0.1:8000`.	Verify `curl.exe -s http://127.0.0.1:8000/api/health` returns `{"status":"ok"}`.
PowerShell execution policy error	Windows group policy blocks unsigned scripts.	Always launch via `Install_BugSentinelAI.bat` or `Start_BugSentinelAI.bat` which pass `-ExecutionPolicy Bypass` automatically.

CHAPTER 16

Security, Privacy & Safety Boundaries

16.1 Security Architecture

- **Target Allowlisting:** The test engine strictly prohibits arbitrary external URL navigation. The `target_id` parameter in `RunCreateRequest` is validated against an allowlist (`shopdemo`). Requests attempting to point the browser to arbitrary public domains or internal cloud metadata IP addresses (e.g., `169.254.169.254`) are rejected with HTTP 400.
- **Path Traversal Immunity:** Static file retrieval routes (`/artifacts/{run_id}/{file}`) sanitize file paths and enforce resolution within `settings.ARTIFACTS_DIR` . Directory traversal attacks (`../../`) are blocked by FastAPI's `StaticFiles` provider.
- **Zero External Data Leaks:** BugSentinel AI executes 100% locally on the host machine. No telemetry, customer form inputs, screenshots, or code artifacts are transmitted to third-party cloud servers or external LLM APIs.
- **Zero Secrets in Version Control:** The codebase uses `.env.example` templates; all database records and process PIDs reside in the gitignored `data/` and `logs/` folders.

CHAPTER 17

System Limitations and Future Roadmap

17.1 Current System Limitations

- **Single Authorized Target:** The current system is tightly integrated with the local ShopDemo e-commerce application. Registering new targets currently requires updating `planner.py`.
- **Deterministic Scenarios:** Tests are predefined in code; autonomous DOM crawling and LLM-driven test script generation are not yet implemented.
- **Single Browser Engine:** Headless Chromium is the only bundled browser runtime. Firefox and WebKit are supported by Playwright but not provisioned by the installer to conserve disk space.

17.2 Future Development Roadmap

- **v1.1 (CI/CD Integration):** Headless CLI runner emitting JUnit XML and GitHub Actions step summaries for automated pull request gating.
- **v1.2 (Multi-Browser Support):** Installer flags to provision Firefox and WebKit for cross-browser parity verification.
- **v2.0 (Autonomous LLM Exploration):** Integrating Gemini / Claude multi-modal models to explore unmapped web applications, identify interactive elements, and autonomously propose synthetic test scenarios.

CHAPTER 18

Technical Glossary of Terms

- **Assertion:** A programmatic boolean condition that must evaluate to true for a test to pass (e.g., confirming that order confirmation text is absent when an invalid email is entered).
- **Bounded Retry:** A verification mechanism that executes a fixed, limited number of attempts (specifically 2) to establish repeatability without masking race conditions.
- **Chromium:** The open-source browser engine developed by Google that powers Google Chrome and Microsoft Edge, driven headlessly by Playwright.
- **Deterministic Test:** A test scenario whose inputs, navigation steps, and assertion rules are explicitly codified to ensure identical preconditions on every execution.
- **Flaky Test:** A test that yields both passing and failing outcomes across consecutive runs without any modification to the application code.
- **Playwright:** A modern Python and Node.js browser automation library providing reliable cross-browser execution, auto-waiting locators, and headless execution.
- **Regression Retest:** Re-executing a specific test scenario linked to an earlier bug finding to prove that a code modification successfully fixed the defect.
- **Reproducible Failure:** A failure classification applied when a scenario fails on both the initial attempt and an immediate verification attempt.

CHAPTER 19

Quick-Start Reference Cheat Sheet

19.1 Essential Commands & URLs

Action / Resource	Command or Location	Expected Behavior
1-Click Install	Double-click `Install_BugSentinelAI.bat`	Provisions .venv, packages, Playwright Chromium, builds frontend, verifies tests.
1-Click Launch	Double-click `Start_BugSentinelAI.bat`	Starts backend & frontend, polls health, opens default browser to dashboard.
Safe Shutdown	Double-click `Stop_BugSentinelAI.bat`	Terminates only tracked BugSentinel AI processes; frees ports 8000 and 5173.
Testing Lab URL	`http://127.0.0.1:5173/#dashboard`	Full operational workspace with scenario trigger, findings drawer, zoom modal.
ShopDemo Target	`http://127.0.0.1:8000/demo/`	Target e-commerce store containing deliberate email checkout bug.
OpenAPI Swagger Docs	`http://127.0.0.1:8000/docs`	Interactive API documentation for all REST routes.
Run Automated Tests	`.\.venv\Scripts\python.exe -m pytest -v`	Executes full 25-test suite (unit, contract, and browser integration).
Package Distribution	`powershell scripts\windows\Package-BugSentinelAI.ps1`	Compiles clean, transferable `BugSentinelAI-Release.zip` (0.17 MB).